Topic-> Python Dictionary (Hash Table) – Internal Working, Complexity & Version Changes

❓ Q1: What is the inner working of a Python dictionary?

Ans:

- Python dictionary is implemented using a **hash table**
- It provides **fast key-value lookup**

Working steps:

- Take key (e.g., "name")
- Compute **hash(key)**
- Convert hash → index using modulo
- Go directly to that index
- Return value

❓ Q2: How does dictionary access work internally?

Ans:

Example:

student["name"]

Steps:

- Compute hash("name")
- Convert to index (hash % table_size)

- Jump directly to memory slot
- Return value

Time complexity: O(1) average

❓ Q3: How does insertion work in dict?

Ans:

student["college"] = "IMS"

Steps:

- Compute hash of key
- Find index
- Store key-value pair

Time complexity: O(1) average

❓ Q4: How does deletion work in dict?

Ans:

del student["city"]

Steps:

- Hash the key
- Find location
- Remove entry

Time complexity: O(1) average

❓ Q5: What is a collision in dictionary?

Ans:

- When two keys give same index

Example:

hash("abc") % 10 = 5
hash("xyz") % 10 = 5

Both want same slot → collision

❓ Q6: How does Python handle collisions?

Ans:

- Uses **open addressing (probing)**

Steps:

- If index is full
- Check next slots:
  - 5 → 6 → 7 → 8 ...
- Store in first empty slot

❓ Q7: Why is dictionary faster than list?

Ans:

List search:

- Linear search
- O(n)

Dict search:

- Direct hash lookup
- O(1)

❓ Q8: Memory vs Speed trade-off?

Ans:

- Dict uses **more memory**
- But gives **faster access**
- List uses **less memory**
- But slower search

❓ Q9: List vs Dictionary internal operations

Ans:

List:

- Access → O(1)
- Search → O(n)
- Insert middle → O(n)
- Delete middle → O(n)

Dict:

- Access → O(1)
- Search → O(1)
- Insert → O(1)

- Delete → O(1)

❓ Q10: What changed after Python 3.7 in dict order?

Ans:

Before Python 3.7:

- Dicts were **unordered**
- No guarantee of order

Python 3.6:

- Ordered in CPython (implementation detail only)

Python 3.7+:

- Dicts are **officially insertion ordered**

❓ Q11: Do we use append() in dictionary?

Ans:

- ❌ No append() in dict

List:

lst.append(10)

Dict:

d["key"] = value

or

d.update({"a": 1})

❓ Q12: What methods are used to add elements?

Ans:

- List → append()
- Set → add()
- Dict → dict[key] = value or update()

❓ Q13: Complexity summary of dictionary

Ans:

Operation	Average Time
Insert	O(1)
Search	O(1)
Delete	O(1)
Iterate	O(n)

❓ Q14: What is the main idea of dictionary internals?

Ans:

- Hashing converts key → index
- Direct memory access
- Collision handled by probing
- Gives very fast performance

❓ Q15: Final one-line difference (List vs Dict)

Ans:

- List → sequential storage + linear search
- Dict → hash table + direct lookup